COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	09/11/2026	**DATE SUBMITTED:**	09/12/26
Members:	Condesa, Andrie Garcia, Dennis James T. Hare, Andrei Angelo D. Ines, D Jay Larracas, Jerry Matthew Pellazar, Stephen Kurt R.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

SEATWORK NO. 8.1 Using Sorting Algorithms 2

1. PROCEDURE OUTPUT

What procedures did you perform?

Objectives: To create a program in C++ using sorting algorithms.

1. To be familiarized with the sorting algorithms.

2. To be able to differentiate the types of sorting algorithms.

3. To be able to create a program with sorting algorithms.

Answer the following questions:

1. Explain the quick sort, shell sort and merge sort types of sorting algorithms.

Quick sort – is a sorting technique that employs "divide and conquer." Choose a number from the list, referred to as the pivot, then shift all smaller numbers to one side of it and all larger numbers to the opposite side. The pivot is now positioned accurately. You continue this identical procedure with the smaller groups until all is organized.It's generally quite swift, but consistently choosing poor pivots can significantly hinder its speed. That's the reason individuals occasionally choose the pivot at random to prevent that issue.

2. **Shell sort** – is an enhanced version of insertion sort where it introduces a gap based variable. How it works is that you calculate a starting value, divide the array's total size by a specific number. Then, if this value contains a decimal, the decimal is ignored, and the remaining whole number is considered the "gap".Which determines the number of indexes that the algorithm will traverse to compare, swap, and sort values. This process is iterated several times, until it closes the gap. When the value of the gap reaches one, it uses a standard insertion sort to compare, swap, and fully sort the remaining values.

3. **Merge Sort** – is a divide and conquer pattern that works by splitting an array into smaller halves, sorting each half, and then merging them back together in order. The process begins by recursively dividing the array until each sub-array contains only one element, which is naturally sorted. Then, the pattern carefully merges these sub-arrays by comparing elements and arranging them in ascending order.

2. Give simple sample programs in C++ that use the above sorting algorithms. Use user input of 10 integer values as the array elements to be sorted. Explain how the programs work.

Quick Sort Program

Pseudocode:

```
quickSort(array, low, high)
        If (low < high)
        pivot = arr[high]
        i = low - 1


        for(j = low; j < high; j++)
                if arr[j] < pivot
                {
                 i++
                temp = arr[ i ]
                arr[ i ] = arr[ j ]
                arr[ j ] - temp
                }


        temp = array[i + 1]
        array[i + 1] = arr[high]
        Array[high] = temp

        pivotIndex = i + 1

        quickSort(array, low, pivotIndex - 1)
        quickSort(array, pivotIndex + 1, high)
```

Explanation: This algorithm begins by selecting the last element in the array as the pivot point to serve as a benchmark. It then iterates through the array, comparing each element against this pivot and swapping smaller values to the left side. Once all elements are compared, the pivot is swapped into its correct index right between the smaller and larger values. Finally, the pseudocode calls itself recursively to repeat this exact partitioning process on the left and right sub-arrays until the whole collection is sorted.

Shell Sort Program

Pseudocode:

```
shellSort(array, size)
        for(gap = size / 2; gap > 0; gap /= 2)
                for(i = gap; i < size; i++)
                {
                 temp = array[ i ]
                 j = i

                while ( j >= gap && array[ j - gap] > temp)
                        {
                         array[ j ] = array[ j - gap]
                        j -= gap
                        }
                   array[ j ] = temp;
                }
```

Explanation: This process starts by setting an initial distance gap equal to half the total size of the array. It then iterates through the array and compares elements that are separated by this gap distance rather than checking adjacent neighbors. If an element on the left is greater than the current item, the algorithm shifts the larger value to the right to make room. After a full pass, the gap size is halved (gap /= 2), and the comparison process repeats until the gap reaches 1, concluding with a final, highly efficient standard insertion sort pass.

Merge Sort Program

Pseudocode:

```
START

function mergesort(A, low, up)
   if low < up
      middle = (low + up) / 2
      mergesort(A, low, up)
      mergesort(A, middle + 1, up)
      merge(A, low, middle, up)

function merge(A, low, middle, up)

END
```

Explaination: This algorithm operates by calculating the middle index using the formula (low + up) / 2 to split the array range into sections. It then uses recursive function calls to divide the array down into smaller sub-problems until individual elements are reached. Once the divisions are complete, it calls the separate merge function to combine the elements back together in proper ascending order.

Main function Pseudo Code

```
array[10]
print " Enter 10 integer values: \n"
for(i = 0; i < 10; i++)
user input >> array[ i ]

Print " User's Entered values in Array:  "
for (i = 0; i < 10; i++)
        print array[ i ]

quickSort( array, 0, 9 )
shellSort( array, 10 )
mergeSort( array, 0, 9 )

prints all  quickSort, shellSort and mergeSort values
```

Explanation:
This shows the main function of the program where the program will have a user input to add values into the array until it reaches the max limit of 10. Then it will display the values that the user inputs inside of the array and it will also display the three types of sorting algorithm including quickSort, shellSort, and mergeSort.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n\a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n\a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n\a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.